GoServer

N-player Go in the spirit of KGS

Project team

November 1, 2026

Contents

1 What GoServer is **1**
- 1.1 Running it . . . 1

2 Rooms, challenges and chat **2**
- 2.1 Navigation . . . 2
- 2.2 Identity . . . 2
- 2.3 Bounds . . . 2
- 2.4 Correlation and replay . . . 2

3 N-player Go rules **3**
- 3.1 Board, turns and captures . . . 3
- 3.2 Losses, bowls and score . . . 3
- 3.3 Ko and repetition . . . 3
- 3.4 Clocks and ending . . . 3
- 3.5 Challenge parameters . . . 4

4 Seats, bots and teams **5**
- 4.1 Seat plan . . . 5
- 4.2 Bots . . . 5
- 4.3 Autopilot . . . 5
- 4.4 Team merges . . . 5

5 What every viewer sees **6**

6 Requirements **7**
- 6.1 Shape . . . 7
- 6.2 Positive invariants . . . 7
- 6.3 Negative requirements and their controls . . . 7
- 6.4 Derived outputs . . . 8

7 Implementation **9**
- 7.1 Evidence . . . 9

8 Usage scenarios **10**
- 8.1 Welcome . . . 10
- 8.2 Creating a challenge . . . 11
- 8.3 Planning seats . . . 12
- 8.4 Reading the bot's search . . . 13

8.5 A watcher explores a variation 14
8.6 Proposing a team merge 15
8.7 Playing as a team, on autopilot 16
8.8 The result 17

9 Known gaps **18**

A Task catalog **19**

B How the project got here **25**

Chapter 1

What GoServer is

GoServer is a local, in-memory online Go service for *multiplayer* Go: two to eight players on one board, each with a colour, taking turns in order. KGS inspires the navigation (rooms, challenges, chat) but there is no compatibility with KGS accounts, protocols or clients. The rules engine is a port of the stakeholder's single-page N-Player Go Lab, whose JavaScript (`source/cmd/demo/static/engine.js`) remains the reference.

A visitor picks a display name, enters a room, talks in room chat and opens or creates a challenge. The creator chooses the board and clocks, decides seat by seat whether a person or a bot plays and who is on which team, and starts the game. Empty seats become bots that search with Monte Carlo tree search on the server. During play a person can hand their seat to a bot (autopilot) and take it back, and teams can merge when everyone on both teams agrees. Every viewer can step through the game's history and see each bot move's search; watchers can branch and run their own searches, which never reach the server.

The record of the project is this book together with `spec/Requirements.lean`, `spec/Requirements.proto`, `spec/Implementation.lean` and the source in `source/`. Review and acceptance happen in git; chat histories are disposable.

1.1 Running it

From the project root:

- `make run` builds and serves the app on `127.0.0.1:8081` (`make run PORT=8090` picks another port; LeanFM's own 8080 is never used); `make stop` stops only the process it started.
- `make spec-check` runs LeanFM's canonical check on the three specification files and regenerates `spec/tasks.json`.
- `make views` derives the task figures of Appendix A from that catalog.
- `make app-check` builds and runs `go test -race`.
- `make browser-check` drives a real headless Chrome through two users' room, challenge and chat flow; `make tour` replays the usage scenarios of Chapter 8 and refreshes their screenshots.
- `make book` rebuilds this book in light and dark copies.

All state lives in memory and resets on restart.

Chapter 2

Rooms, challenges and chat

2.1 Navigation

After choosing a name a person sees the rooms (the lobby always exists) and can create a room. Entering a room shows its conversation and its challenges. Opening a challenge shows its seats, its board once started, and its own chat. Room chat and game chat are separate conversations: nothing posted in one is copied to the other, and only members of a room or game receive its chat. Watching or joining a game enters its chat; neither grants any move authority.

2.2 Identity

The server issues the session; creator and sender are always derived from it, never taken from request fields. There is no account registration in this local build: the display name identifies a person for the life of the server.

2.3 Bounds

Names are 1–128 Unicode characters, trimmed; room names are unique and case-sensitive and are never interpreted as paths or markup. Chat bodies are 1–4000 characters, and each room or game keeps its latest 100 entries. There are at most 16 rooms including the lobby and 64 challenges in total. Each session has a finite budget of admitted mutations; once it is spent, new mutations are rejected while already accepted results stay replayable, so a retried request never applies twice.

2.4 Correlation and replay

Every request carries a request ID unique per session and task, and receives exactly one result: its success message or `RequestRejected`, with the same ID. A duplicate mutation returns the earlier result; reusing an ID with a different body is rejected. A rejected or overloaded request changes nothing and discloses no other person's identity or session.

Chapter 3

N-player Go rules

3.1 Board, turns and captures

The board has 5, 7, 9, 11, 13 or 19 lines and is bounded by a Wall. Seats move in order. After a stone is placed, every adjacent group of *another* colour left without liberties is removed first. Only if nothing was captured and the placed stone's own group has no liberties is the move a suicide, and then that whole group is removed. Suicide is legal: in multiplayer Go a player may sacrifice a group to help an ally make eyes.

3.2 Losses, bowls and score

A removed stone counts against its *owner*, not for the capturer: it goes to the owner's losses, and the bowl records which seat captured it (suicide is recorded against the mover). Territory is an empty region bordered only by one colour's stones and the Wall. A seat's score is its territory minus its losses; a team's score is the sum of its members' territory minus the sum of their losses,

$$S_T = \sum_{p \in T} A_p - \sum_{p \in T} G_p.$$

The winners are the team or teams with the highest score among teams that still have a member who has not resigned.

3.3 Ko and repetition

The ko rule is a challenge parameter: *Auto* (the default: on for two players, off for three or more), *Off*, or *On*. When on it is simple ko plus *situational* superko: a stone may not recreate an earlier board with the same seat to move. The Lab calls this "positional superko", but its comparison includes the player to move, so situational is the accurate name. A Zobrist hash only indexes candidates; equality is decided on the exact board and seat, so a hash collision can never reject a legal move. Passes, timeouts and resignations are never rejected for repeating a position.

3.4 Clocks and ending

A person's turn has a deadline (5 seconds to 64 minutes, default 10 seconds); a bot's search has its own budget (0.25 to 16 seconds, default 1 second). An unanswered turn becomes a *timeout*: it

passes the turn but neither ends nor resets a round of passes. A move arriving after its deadline is rejected and the timeout wins. The game ends when every remaining seat has passed since the last stone, after one full round of timeouts (scored as it stands), or at $3N^2$ moves on an $N \times N$ board. A resigned seat passes automatically and cannot win. The final scores and winners are announced in game chat.

3.5 Challenge parameters

Board size (default 11), seats 2–8 (default 4), the two clocks, the ko rule, and the bot settings of the Lab: exploration constant 0.5, 0.7 (default), 1.0 or 1.4; reward *margin* (a soft margin against the best rival, the default), *win* or *rank*; and margin softness 2, 4 (default) or 8 points. Zero or empty means the default; anything out of range is rejected.

Chapter 4

Seats, bots and teams

4.1 Seat plan

Before the start the creator plans each seat as *bot* or *open* and gives it a team number from 1 to the seat count; by default each seat is its own team. People in the room take open seats (one seat per person per game) or leave them. A seat planned as a bot cannot be taken, and a seat a person holds cannot be turned into a bot seat until that person leaves. Only the creator starts the game; every seat still without a person becomes a bot named `bot-`*seat*. Seats never change after the start.

4.2 Bots

Bots run Monte Carlo tree search on the server, for their team's combined score. A bot turn is dispatched to a bounded pool of search workers (at most $\min(8, \mathit{CPUs}/2)$ at once), each with a time budget and a 200,000-node cap, and the search never holds the game lock. A result is applied only to the game, seat and turn revision it was dispatched for, and only before that turn's deadline; a late result is discarded (Figure A.18). No client can submit a bot's move, and a bot never moves for a person's seat. Each bot move publishes its search: playouts, time, the top candidates with visit counts, the mover's expected reward, and every seat's projected final score and outlook.

4.3 Autopilot

A seated person may switch autopilot on, letting a server bot play their seat with the bot clock and published search, and switch it off at any time. The seat, team and name stay theirs. Switching off during their own turn discards the pending bot result and gives them the rest of the human deadline, measured from the start of the turn.

4.4 Team merges

During play a seated person may propose merging their team with another team that also has a seated person (nobody could consent for an all-bot team). The merge happens once every person seated in either team accepts; it takes the smaller team number and is permanent. Any decline, any change to either team, or the game finishing withdraws the proposal; there is at most one pending proposal per pair of teams. Bots of the merged team then search for its combined score (Figure A.20).

Chapter 5

What every viewer sees

The browser reproduces the Lab against server state:

- the board with Wall coordinates, territory squares, the last-move marker, ghost markers for next moves and a search heat overlay;
- navigation with Home, Back, Forward, End and previous/next variation, by buttons and by the arrow, Home and End keys; a variation tree and next-move chips;
- a *Search at this position* panel listing candidate moves with visit share and reward, from the bot's published search or the viewer's own;
- per seat: Territory, Bowl, Score, Proj. (projected final score) and Outlook (expected reward), and team totals;
- a clickable projected-score timeline along the viewed line;
- pending merge proposals with who has accepted and who must still answer;
- GIF export of the viewed line, and stone, capture and last-ten-seconds sounds with a toggle;
- live following while at the latest move, otherwise a notice that new moves arrived.

Variations and local searches run in the viewer's browser with the Lab engine on a copy of the position. They are never sent to the server, never change the game, and vanish when the page closes. Watchers may branch and search at any time; a person seated in a game in play may browse the actual moves, and may branch and search once the game has finished. That last rule is a fairness convention of the provided browser, not a security boundary: a public position can always be copied into another tool. The page is dark-only.

Chapter 6

Requirements

6.1 Shape

`spec/Requirements.lean` declares two actors that people see, `Person` and `Server`, and a third, `Bot`: the bounded pool of server search workers. Every operation is a task whose grammar is a request followed by its correlated success or `RequestRejected`. Five tasks are composed: `bot_turn`, `turn` (a person's move, a bot turn, a timeout or an automatic pass), `team_merge`, and `game_lifecycle`: create, then any number of seat, join, chat and state operations, then start, then turns, autopilot switches, merges, chats and state reads, until `GameFinished` (Figure A.21). Server-originated events (`BotMovePlayed`, `TurnTimedOut`, `AutoPassed`, `TeamsMerged`, `MergeWithdrawn`, `GameFinished`) reach game members as entries of `GameState`, and `GetGameState` takes `since_move` so a viewer fetches only new move records. `spec/Requirements.proto` declares the same messages.

6.2 Positive invariants

Every room, chat, board, clock, score and team a viewer sees comes from one server-authoritative transition in a scope the viewer is authorized for. Every turn ends exactly once ($AG(\mathit{to_move} \Rightarrow AF\ \mathit{moved})$), and every started game finishes ($AG(\mathit{playing} \Rightarrow AF\ \mathit{finished})$).

6.3 Negative requirements and their controls

Each *never* obligation of the specification is held by a control and a test; `source/traceability.json` lists them in full.

- **A move for another seat, out of turn, or for an autopilot seat**: seat-to-move identity check under the game lock (`TestMovesAreAuthorizedOrderedAndRuleChecked`).
- **Two transitions for one turn** (move, bot result, timeout, auto-pass): a turn sequence number under the lock and the replay cache (`TestTurnsNeverEndTwice`).
- **A move accepted after its deadline**: the deadline is rechecked on arrival and the timeout applied first (`TestLateMoveLosesToTheDeadline`).
- **A stale or misdirected bot result**: results bound to game, seat and turn revision (`TestBotNeverMovesFor`, `TestAllBotGamePlaysToTheEnd`).

- **A legal move rejected by a hash collision**: exact situation compared behind the Zobrist key (`TestSituationalSuperkoUsesExactPositions`).
- **Unbounded bot work**: search outside the lock, time budget, node cap, bounded slots (race-enabled `TestAllBotGamePlaysToTheEnd`).
- **A merge without consent**: per-person acceptance sets and withdrawal on any team change (`TestMergesNeedEveryPersonsConsent`, `TestTeamChangeAndFinishWithdrawProposals`).
- **Seat or plan changes by the wrong person or after the start**: creator and status checks (`TestSeatPlanIsTheCreatorsBeforeStart`, `TestSeatsStartAndBotsFillEmptySeats`).
- **Crossing a room or game boundary**: membership checks on every read and write (`TestNamedChallengesAndChatHistoryStayInScope`).
- **Executable user text, fabricated destinations, stale responses replacing another scope, exposed tokens**: text-only DOM rendering, selection-generation guards, strict bounded JSON, Host and Origin checks (`TestMethodOriginHostAndBodyRejections`, `browser_check.py`).
- **A game that never ends**: passes, the timeout round and the $3N^2$ cap (`TestPassesEndTheGame`, `TestDeadlineTimesOutAndAbandonmentEnds`).

6.4 Derived outputs

Besides the task views, the specification asks for two outputs: the projected-score timeline (per seat, against move number, from bot searches and the viewer's local searches), and the game result (team scores and winners).

Chapter 7

Implementation

`spec/Implementation.lean` maps the specification onto Go. The server (`source/server.go`, `source/game.go`) holds rooms, games and chat in memory behind per-game locks; `source/engine.go` ports the Lab's rules and max-n Monte Carlo search, extended to return the Lab's search summary; `source/bot.go` is the bounded worker pool that realizes the `Bot` actor; `source/channel.go` is the bounded queue whose full sends block until space frees, as the requirements' channel semantics demand. The browser (`source/cmd/demo/static/`) serves the Lab engine and search worker as same-origin files, so viewers can branch and search locally under a content security policy that adds only same-origin workers and `blob:`/`data:` images for GIF preview.

The HTTP demo in `source/cmd/demo/` carries the protobuf-shaped messages as bounded JSON. It is an inspection harness for the server core, not evidence of protobuf byte conformance and not a production account system.

7.1 Evidence

- `reports/specification-check/validation.json`: the canonical check of the three specification files, with their hashes.
- `go test -race ./...` in `source/`: unit, concurrency and negative tests named above.
- `reports/browser/`: the two-user Chrome check and its screenshot.
- `book/assets/tour/manifest.json`: the scripted usage tour, with image hashes, source hashes and browser version.

A passing check establishes elaboration, protobuf agreement, structural completeness and the provenance of the figures. It does not prove that the running server satisfies the temporal, isolation, concurrency or resource obligations; those rest on the controls and tests above.

Chapter 8

Usage scenarios

These screenshots come from the running application, driven by a script that plays the parts of three people: alice creates the challenge, bob joins her, and carol watches. `make tour` in the project root reproduces them; `book/assets/tour/manifest.json` records each step, viewer, image hash, source hashes and browser version. Bot moves come from a time-limited search, so the stones differ between runs while the steps do not.

8.1 Welcome

A visitor chooses a local display name. Nothing else is needed for the local preview.

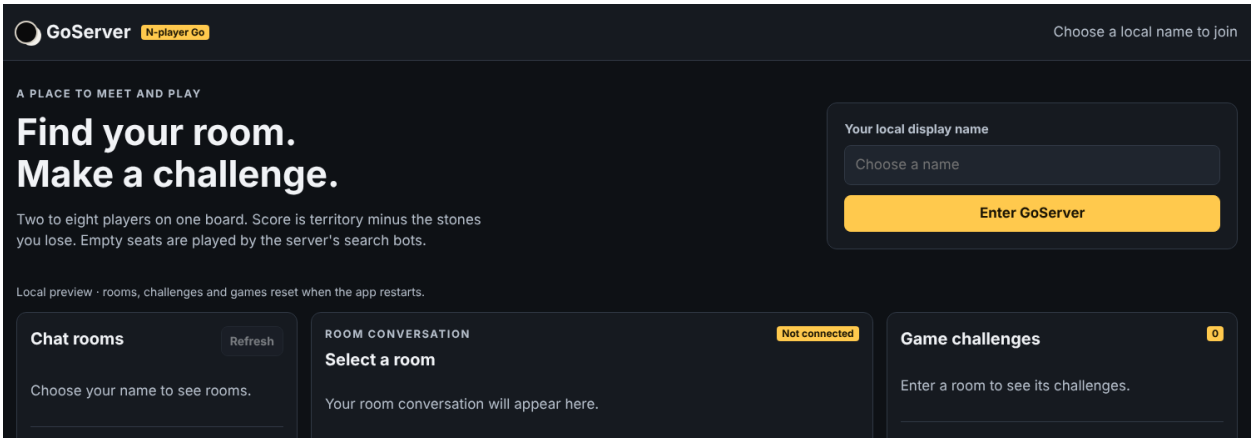


Figure 8.1: Welcome, seen by alice. Steps: open the app.

8.2 Creating a challenge

Alice is in the lobby and fills in a challenge: a 9×9 board, three players, a one-minute human deadline and half a second per bot move. The ko rule stays on Auto, which is off for three players.

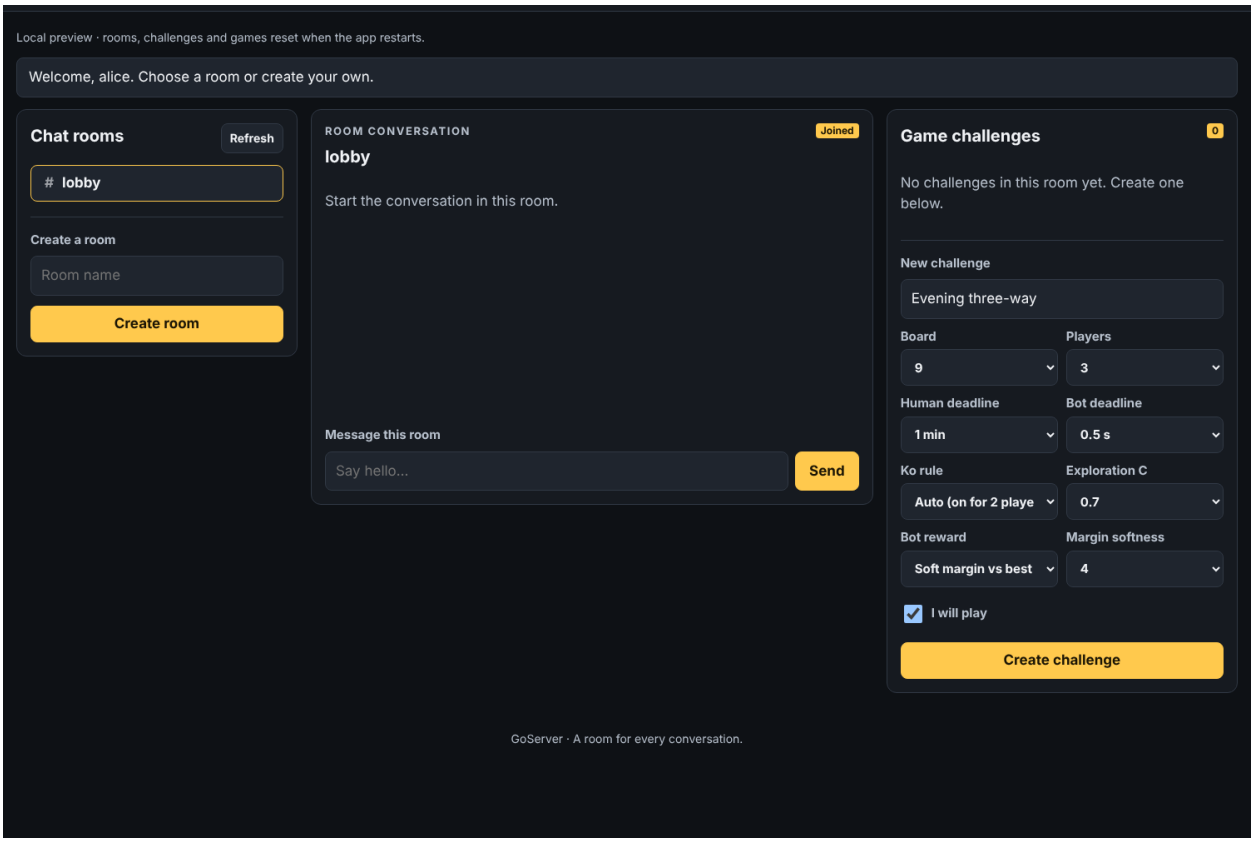


Figure 8.2: Creating a challenge, seen by alice. Steps: sign in as alice; fill in the New challenge form.

8.3 Planning seats

Before the start the creator decides, seat by seat, whether a person or a bot plays and which team the seat is on. Alice took seat 1, bob sat in seat 2, and alice marked seat 3 as a bot.

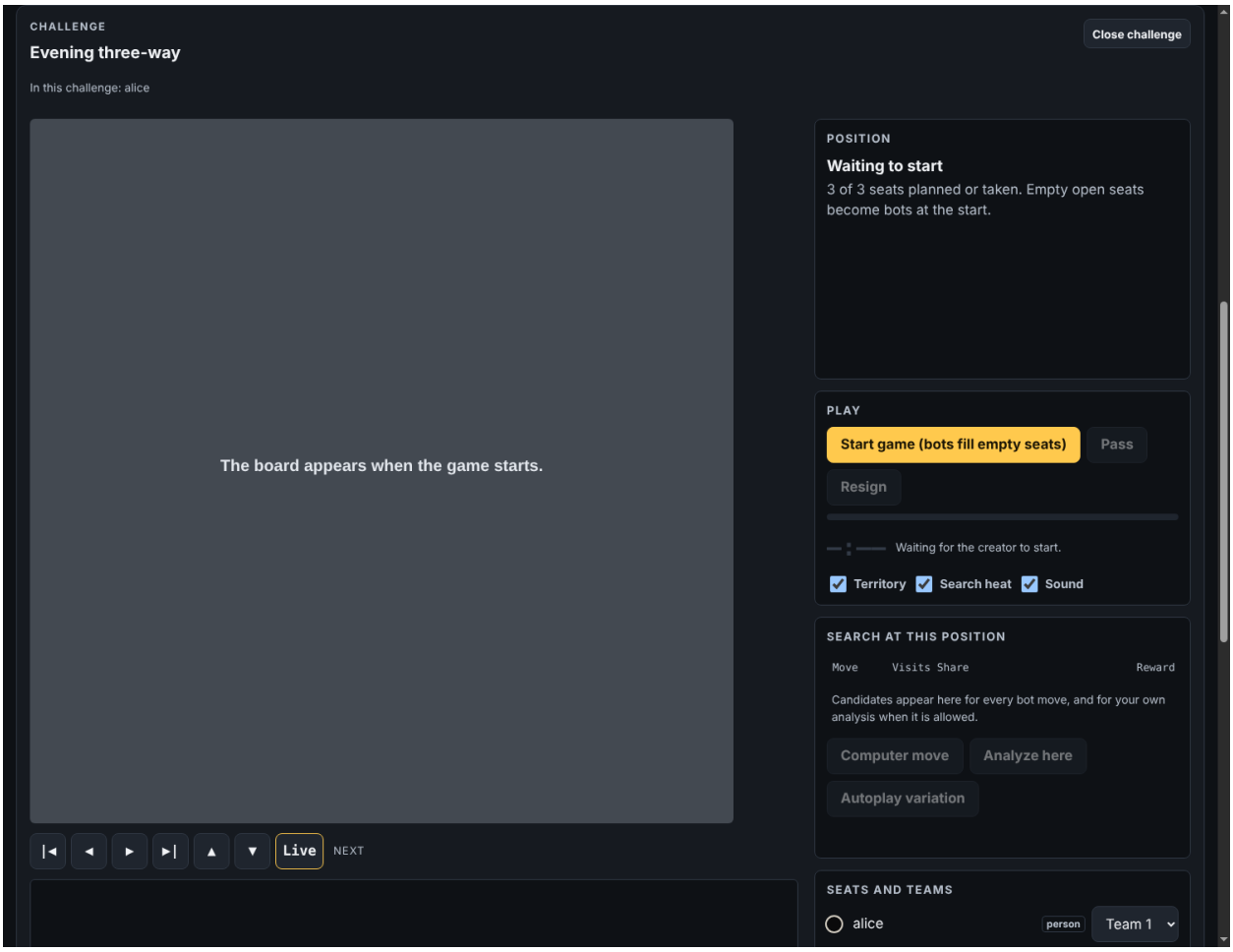


Figure 8.3: Planning seats, seen by alice. Steps: alice: create the challenge (seated automatically); alice: set seat 3 to Bot; bob: open the challenge, Sit here.

8.4 Reading the bot's search

Alice steps back one move, to the position where the bot searched for its last move. The panel lists the moves that search weighed, with visit share and expected reward; the blue discs on the board are the same search as a heat map. Proj. and Outlook give every seat's projected final score and expected reward, and the timeline under them plots the projections move by move.

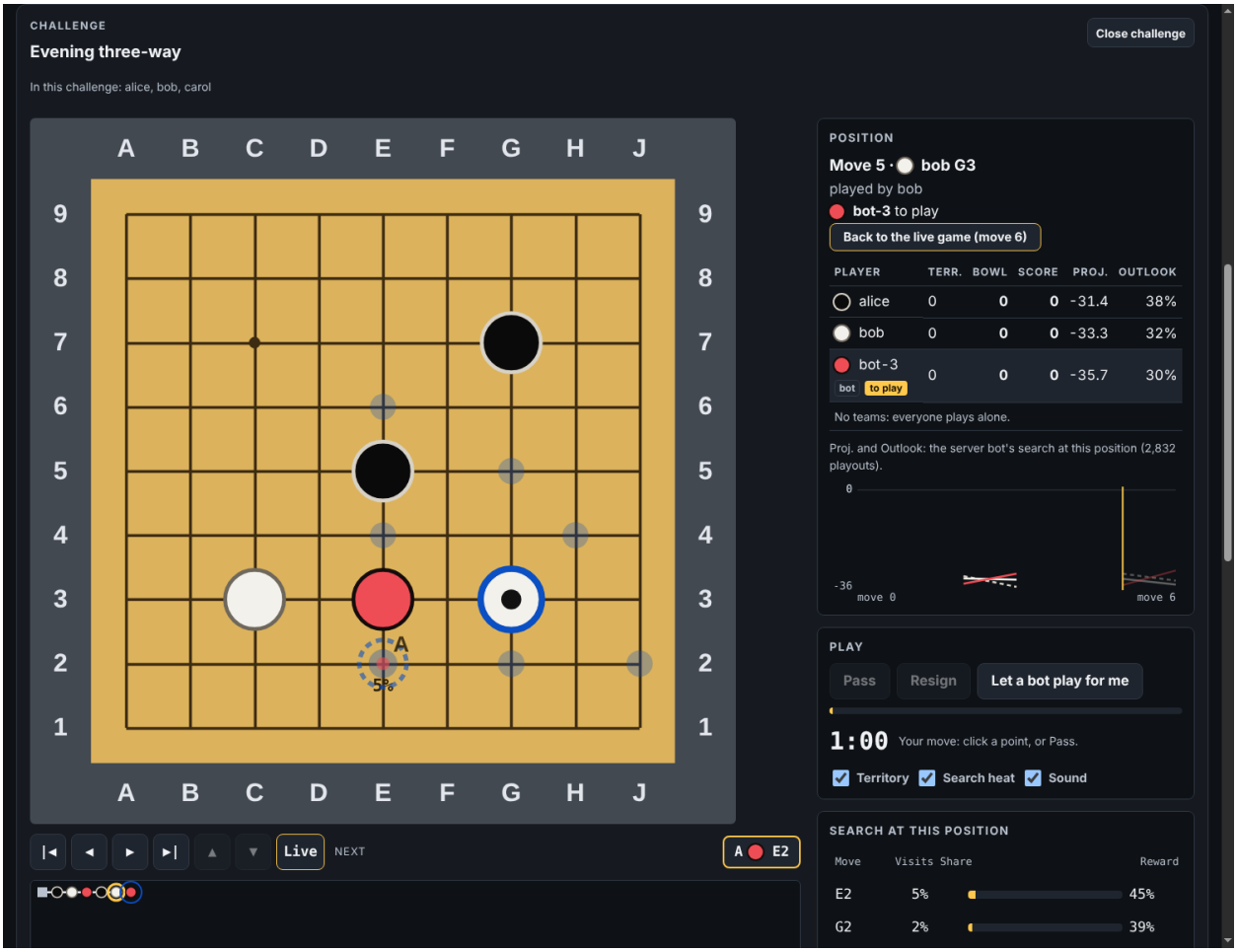


Figure 8.4: Reading the bot's search, seen by alice. Steps: alice and bob alternate stones by clicking the board; the bot answers each round; alice: Back one move.

8.5 A watcher explores a variation

Carol is watching, not playing. She steps back to move 3, tries a different stone (the gold ring marks a variation), and asks for a local search. The analysis runs in her browser only: the dashed branch in the tree never reaches the server, and the live game continues.

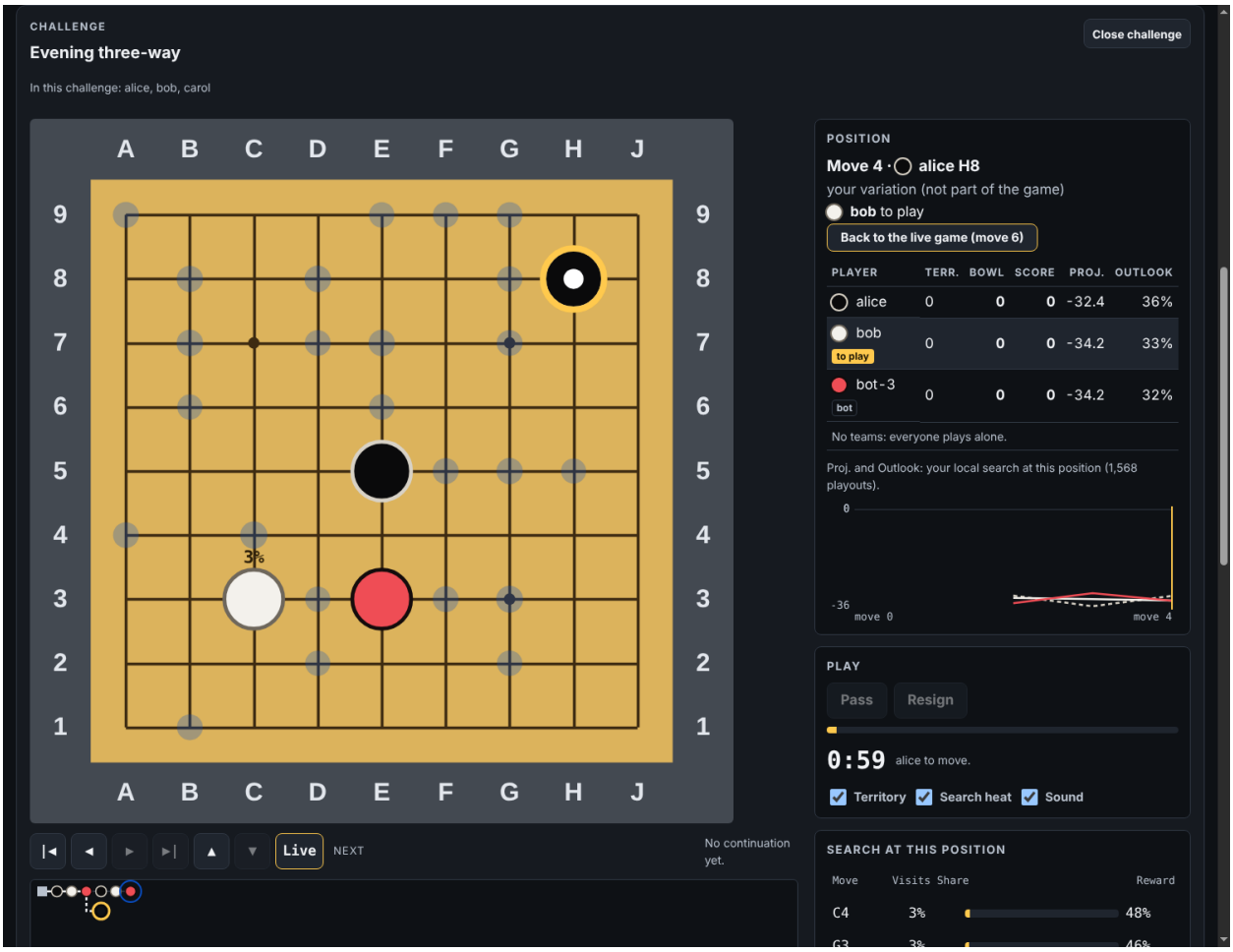


Figure 8.5: A watcher explores a variation, seen by carol. Steps: carol: open the challenge as a watcher; carol: Home, then Forward three times; carol: click an empty point; carol: Analyze here.

8.6 Proposing a team merge

During play bob proposes merging his team with alice's. The proposal shows who has accepted and who is still to answer; it takes effect only when every person on both teams accepts, and it is permanent.

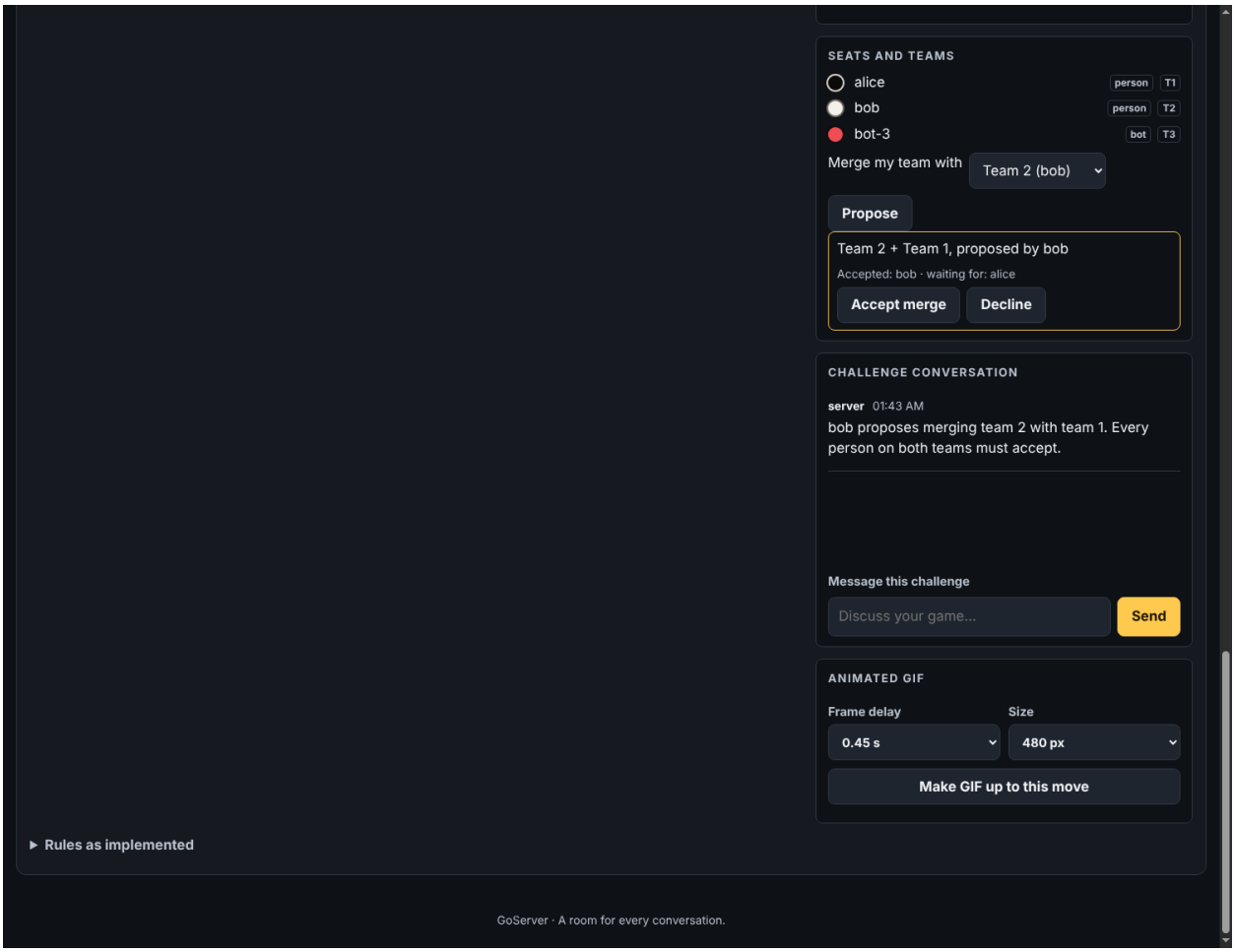


Figure 8.6: Proposing a team merge, seen by alice. Steps: bob: Merge my team with Team 1, Propose.

8.7 Playing as a team, on autopilot

After the merge the score table shows alice and bob as team 1 with a combined total. Alice then lets a bot play her seat; the seat, team and name stay hers, and the bot's move publishes its search like any other bot move. She can take the seat back at any time.

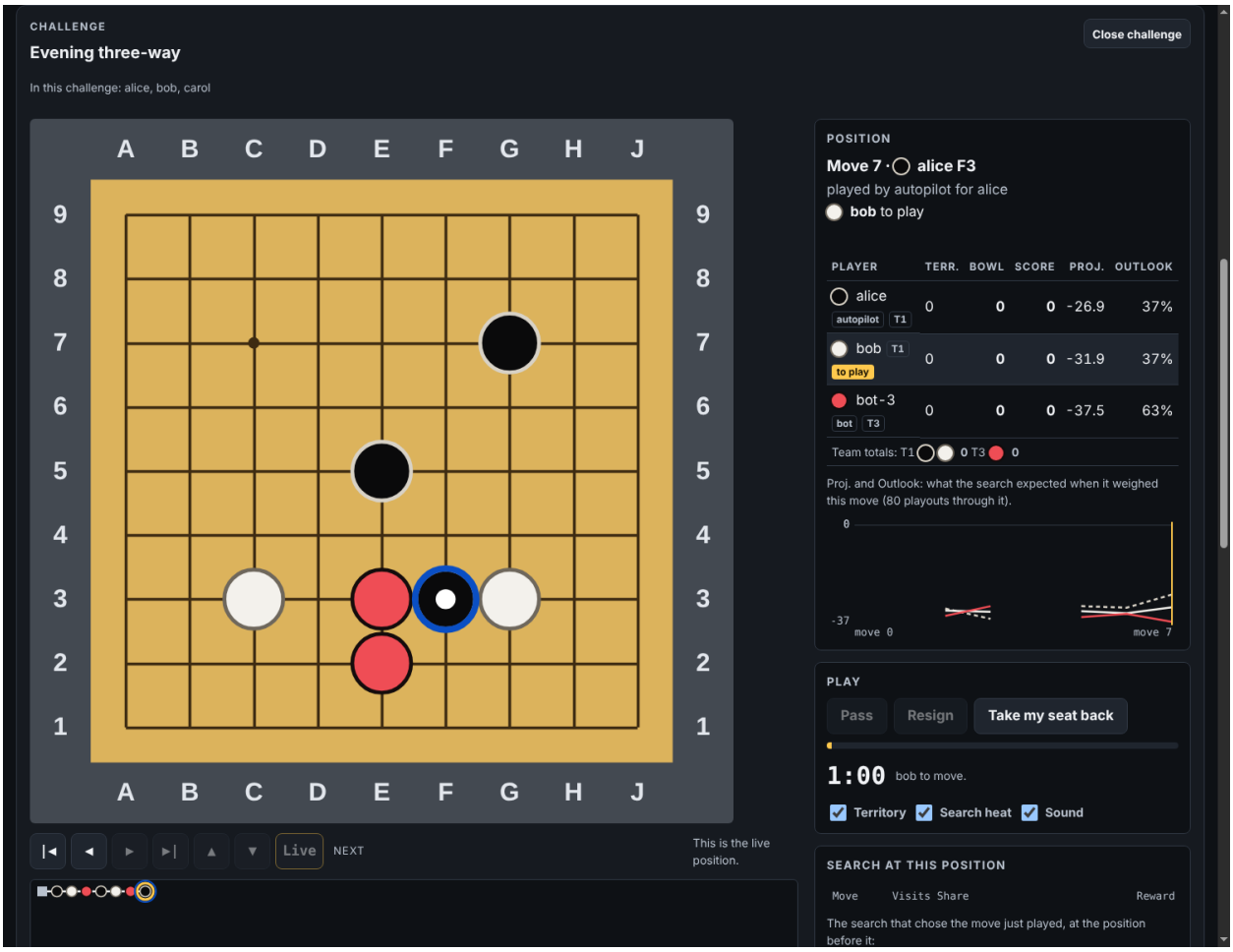


Figure 8.7: Playing as a team, on autopilot, seen by alice. Steps: alice: Accept merge; alice: Let a bot play for me.

8.8 The result

When every remaining player has passed in a row the game ends. The server announces team scores and the winners in the game conversation. Everyone, players included, can now branch and analyse any position.

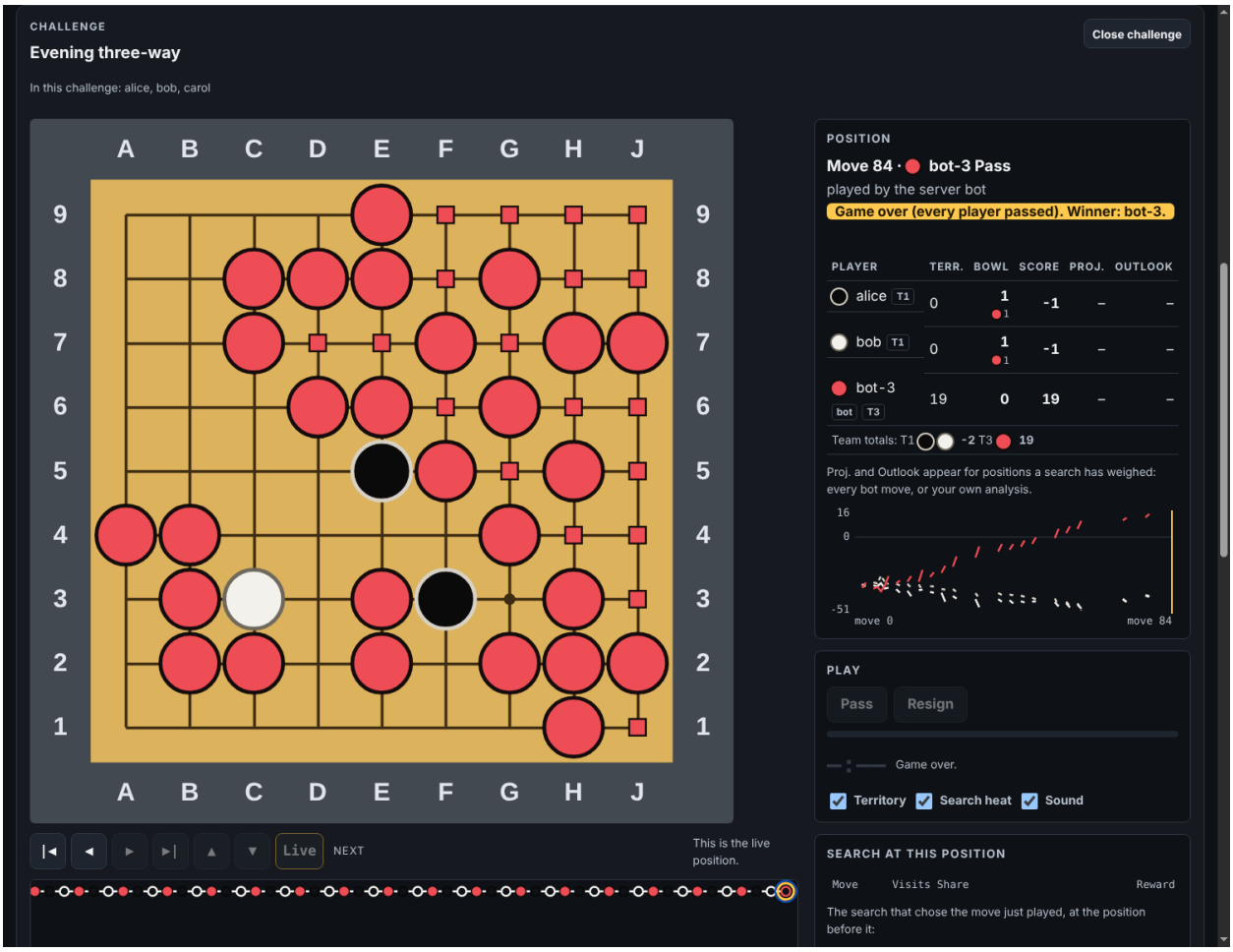


Figure 8.8: The result, seen by alice. Steps: alice and bob pass until the bot also passes.

Chapter 9

Known gaps

This chapter is the work queue. When an item is closed, remove it here and add its evidence under `reports/`.

- Handicaps, ratings and tournaments.
- Persistence across restarts, and SGF export of game records.
- Push transport: the browser polls (every 0.5 seconds while playing).
- Reconnect grace beyond reloading the page, and moderation.
- Accounts: the local build trusts a chosen display name for the life of the server.
- Protobuf byte conformance of the JSON harness.
- Server-side enforcement of the analysis rule for seated players, which is impossible by nature for public positions.

Appendix A

Task catalog

Every task of the checked requirements, as the state machine LeanFM derives from its grammar. Double borders mark terminal states. Regenerate with `make views`.

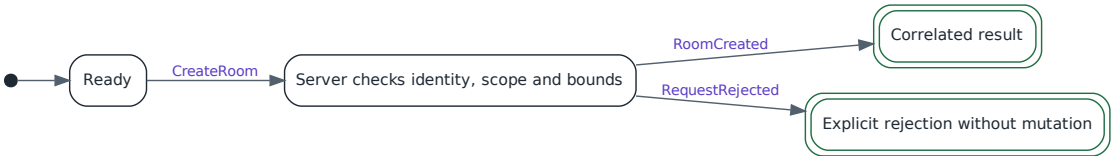


Figure A.1: `create_room`: Create a named chat room

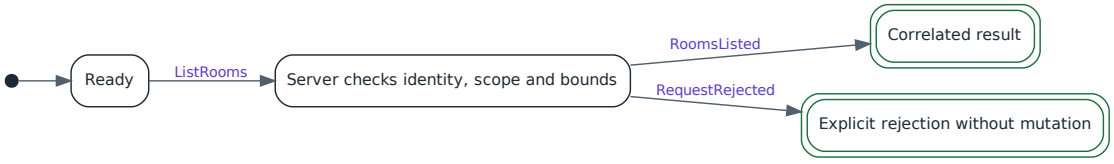


Figure A.2: `list_rooms`: List the available chat rooms

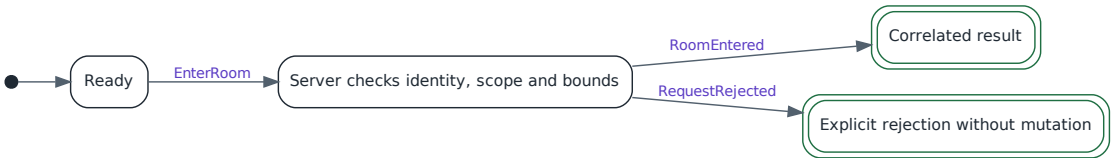


Figure A.3: `enter_room`: Enter a room and inspect its game list

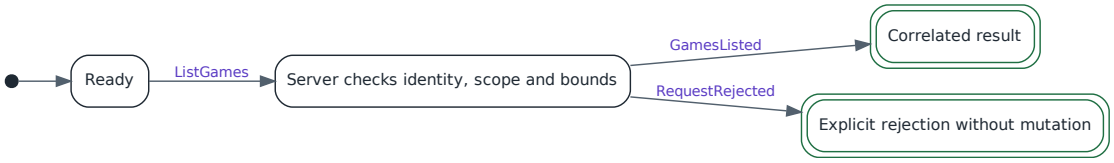


Figure A.4: `list_games`: List room games and open challenges



Figure A.5: `join_game`: Watch or enter a game chat container

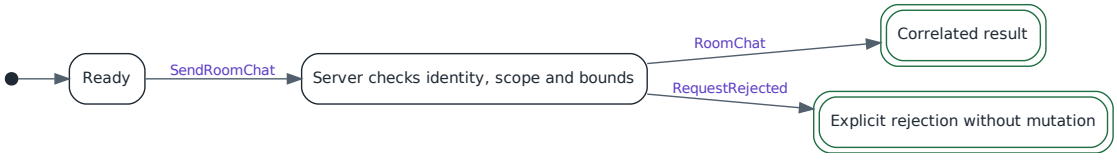


Figure A.6: `room_chat`: Send room chat

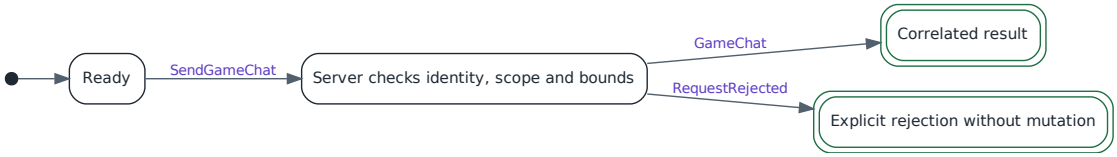


Figure A.7: `game_chat`: Send game chat

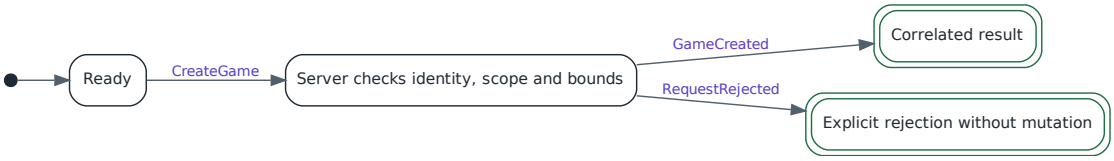


Figure A.8: `create_game`: Create a challenge with board, bot and seat settings

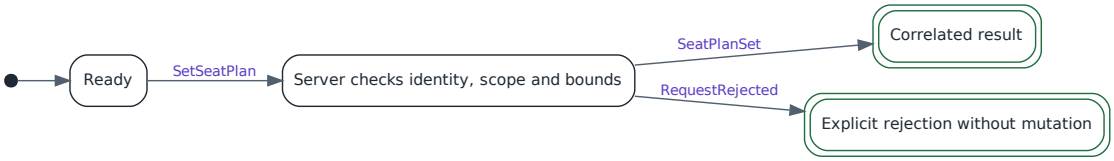


Figure A.9: `set_seat_plan`: Plan a seat as bot or open and set its team



Figure A.10: `take_seat`: Take an open seat

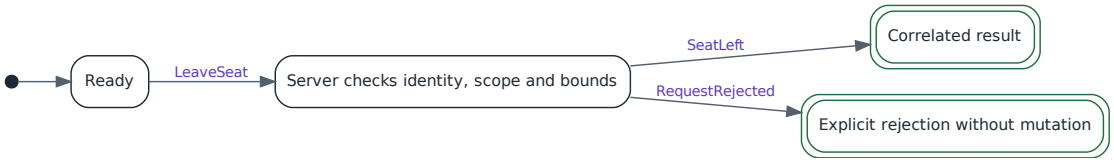


Figure A.11: `leave_seat`: Leave a seat before the start

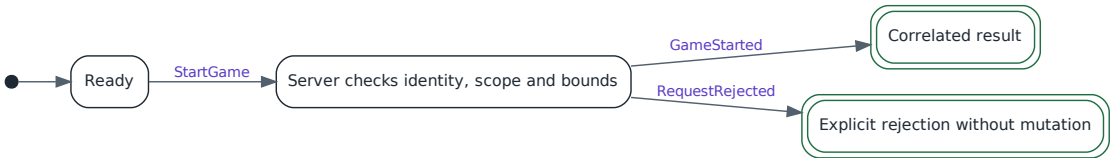


Figure A.12: `start_game`: Start a game; bots fill the remaining seats

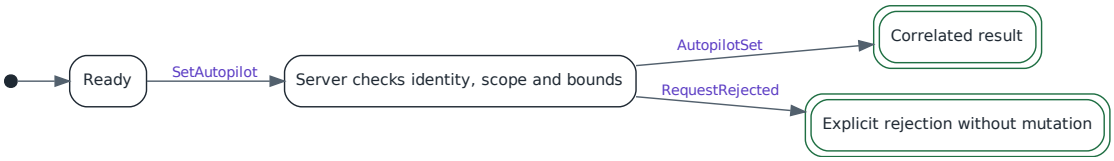


Figure A.13: `set_autopilot`: Hand my seat to a bot, or take it back



Figure A.14: `play_move`: Play a stone, pass or resign

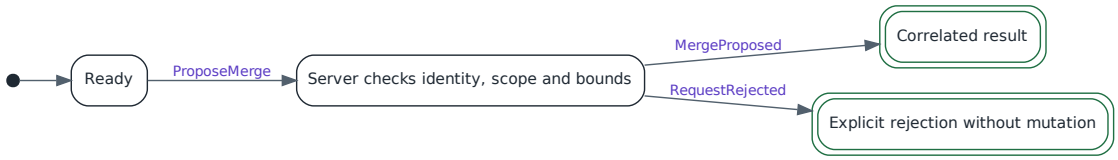


Figure A.15: `propose_merge`: Propose merging two teams

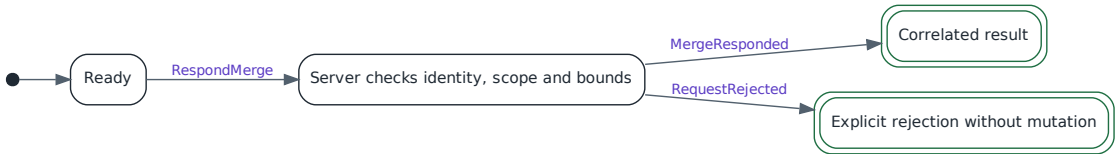


Figure A.16: `respond_merge`: Accept or decline a merge proposal

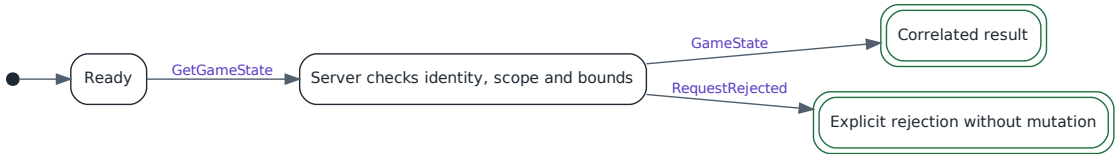


Figure A.17: `game_state`: Watch the board, clocks, scores, teams and record

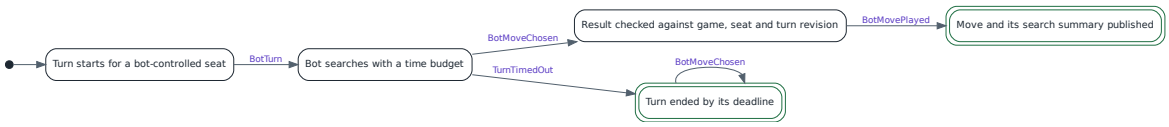


Figure A.18: `bot_turn`: A server bot plays one turn

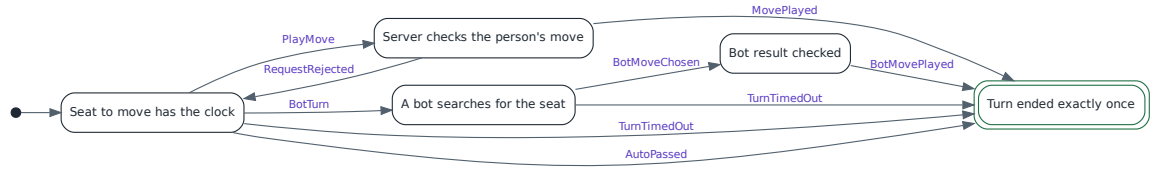


Figure A.19: `turn`: One turn of a started game

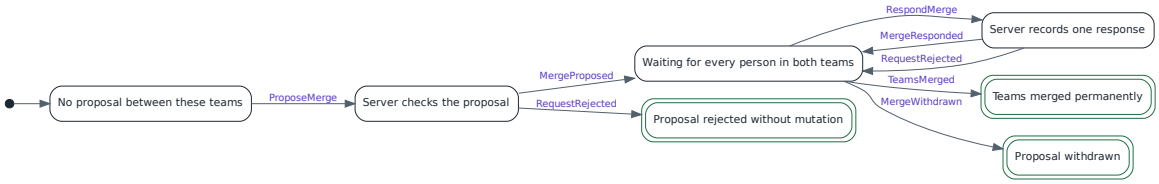


Figure A.20: `team_merge`: Two teams merge by consent

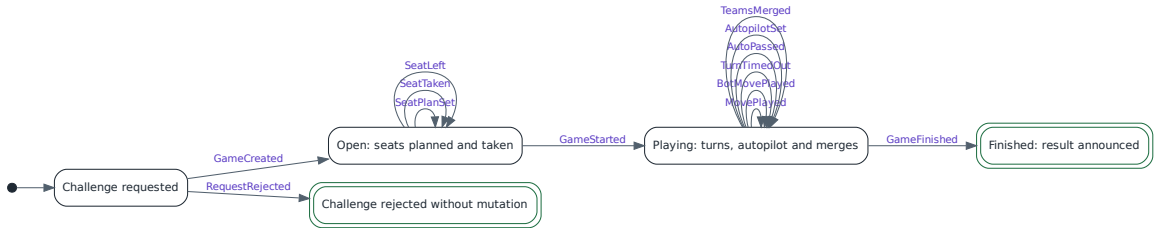


Figure A.21: `game_lifecycle`: A game from challenge to result

Appendix B

How the project got here

The git history is the authoritative log. In outline: the first slice was a server-only room, challenge and chat service with typed correlation and replay; working room creation and a real browser followed; then N-player play with server bots, deadlines and scoring; then parity with the N-Player Go Lab (seat plans, autopilot, teams and merges, search summaries, history, variations and local analysis). On November 1, 2026 the project moved to the repository's current conventions: review in git instead of recorded approval gates, a project-level Makefile, figures derived from the checked catalog, and this book rewritten to describe the system as it is.